MOVEL AI

Take-Home Assignment

Movel AI — ROS Plugin Integration

Overview

This take-home assignment tests your ability to integrate a robot-side ROS system with a cloud-side web application.

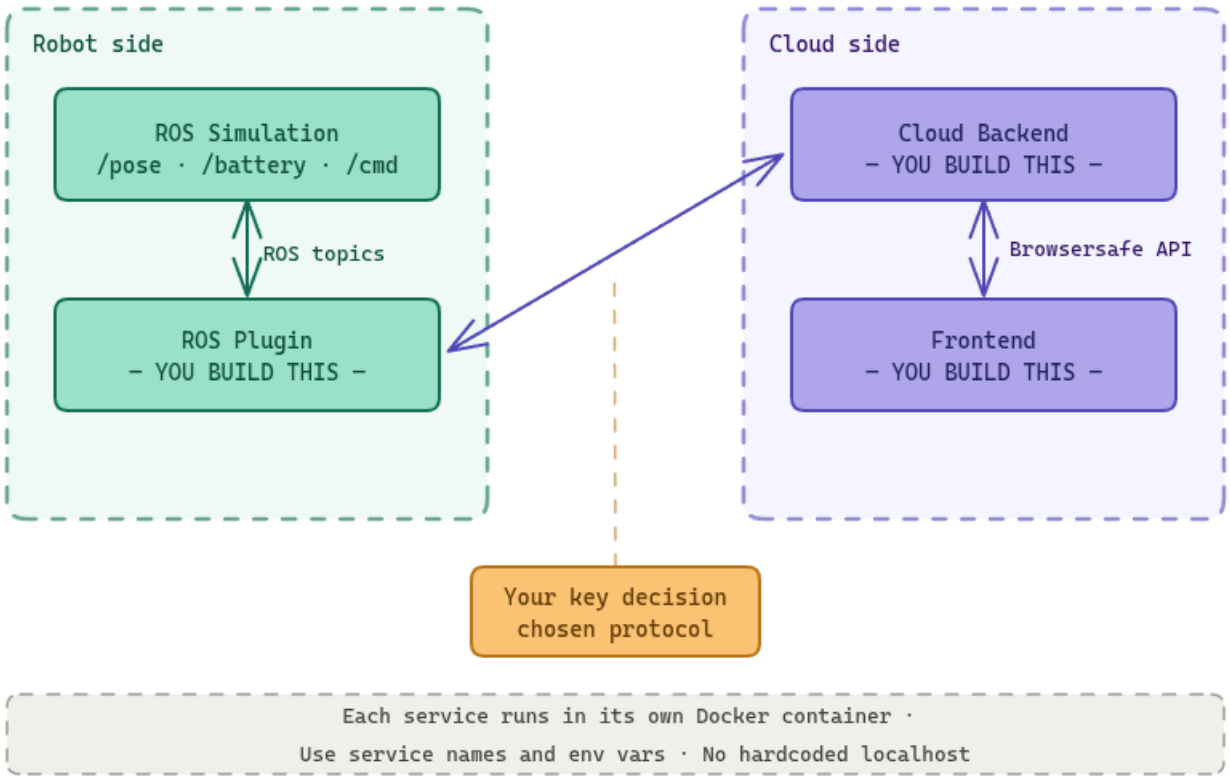
At Movel AI, real robots often run ROS locally on the robot computer, while the product backend and frontend run separately in a cloud or server environment. Your task is to build a small version of that architecture.

You will be given a ROS simulation. You must build:

- A ROS Plugin that runs near the robot and communicates with ROS topics.
- A Cloud Backend that receives robot telemetry and forwards user commands.
- A Frontend that displays robot's state and allows keyboard control.

The goal is not to build a beautiful UI. The goal is to show that you understand service boundaries, robot-to-cloud communication, Docker networking, and reliable state flow.

System Architecture



For this assignment, everything may run on your local machine using Docker, but your design should treat the ROS Plugin and the Cloud Backend as separate systems that could run on different machines.

Important constraints:

- Do not connect the frontend directly to ROS.
- Do not make the cloud backend read ROS topics directly.
- The ROS Plugin must be the only service that talks to ROS.

Provided ROS Simulation

Use the provided ROS simulation from the original Movel AI assignment.

Download the docker compose file from:

> https://drive.google.com/file/d/1SK4sJs7LDxtuAoawysGnm0pXmd1yLQUE/view?usp=share_link

Start the simulation:

```
docker compose -f docker-compose-assignment.yaml up -d

# Or with legacy compose command:
docker-compose -f docker-compose-assignment.yaml up -d
```

Enter the ROS container shell:

```
docker exec -it web-ros bash
```

ROS API Documentation

Robot Position

The /pose topic publishes the robot position every second. Use only x and y for this assignment.

```
rostopic echo /pose
```

Example output:

```
x: 0.0
y: 0.0
z: 0.0
---
x: 0.0
y: 0.0
z: 0.0
---
```

Battery Percentage

The /battery_percentage topic publishes the robot battery percentage. This is mock data.

```
rostopic echo /battery_percentage
```

Example output:

```
data: 50.0
---
data: 50.0
---
```

Robot Movement Command

The /cmd topic accepts a string command using the format data: '<key>', where <key> is one of w, a, s, or d.

```
rostopic pub /cmd std_msgs/String "data: 'w'"
rostopic pub /cmd std_msgs/String "data: 'a'"
rostopic pub /cmd std_msgs/String "data: 's'"
rostopic pub /cmd std_msgs/String "data: 'd'"
```

Key	Direction
W	Up
A	Left
S	Down
D	Right

You may use any ROS client library you are comfortable with:

- roslibjs: http://wiki.ros.org/roslibjs
- roslibpy: https://roslibpy.readthedocs.io/en/latest/index.html

What You Need to Build

1. ROS Plugin

The ROS Plugin represents the software running on or near the robot.

Requirements:

- Runs as its own Docker container.
- Connects to the provided ROS simulation.
- Subscribes to /pose.
- Subscribes to /battery_percentage.
- Publishes movement commands to /cmd.
- Sends the latest robot telemetry to the Cloud Backend.
- Receives movement commands from the Cloud Backend.
- Handles reconnects or temporary communication failures gracefully.
- Exposes clear logs so we can see when it connects to ROS and the backend.

The telemetry sent to the backend must include at least:

```
{
  "robot_id": "robot-1",
  "position": { "x": 0.0, "y": 0.0 },
  "battery_percentage": 50.0,
  "timestamp": "2026-06-03T00:00:00.000Z"
}
```

The movement command received from the backend must include at least:

```
{
  "command": "w"
}
```

Valid commands are w, a, s, and d only.

2. Cloud Backend

The Cloud Backend represents a server that could run on a cloud VM such as AWS EC2.

Requirements:

- Runs as its own Docker container.
- Does not connect directly to ROS.
- Receives telemetry from the ROS Plugin.
- Stores the latest robot state in memory.
- Provides an API for the frontend to fetch the latest robot state.
- Provides an API for the frontend to send movement commands.
- Forwards movement commands to the ROS Plugin.
- Validates commands before forwarding them.
- Provides a health endpoint.
- Uses environment variables for all URLs, ports, and service addresses.

Minimum backend API expected by the frontend:

Method	Endpoint	Description
GET	/health	Backend health and plugin connection status
GET	/api/robot/state	Latest robot position, battery, and last update time
POST	/api/robot/command	Send movement command: { "command": "w" }

Example /health response:

```
{
  "status": "ok",
  "plugin_connected": true,
  "last_telemetry_at": "2026-06-03T00:00:00.000Z"
}
```

You may choose the communication method between the Cloud Backend and the ROS Plugin. Examples include HTTP polling, REST callbacks, WebSocket, Server-Sent Events, message queues, or another protocol you consider appropriate.

You must explain your choice in the README, including the trade-offs.

3. Frontend

The frontend represents the operator UI.

Requirements:

- Runs as its own Docker container.
- Uses TypeScript.
- Runs in the browser.
- React (preferred), or similar frameworks are allowed but not required.
- Displays the robot position on a 2D canvas or equivalent visual area.
- Displays the robot battery percentage.
- Captures keyboard input for W, A, S, and D.
- Sends movement commands to the Cloud Backend, not directly to the ROS Plugin.
- Shows loading and error states for API failures.
- Shows a stale or disconnected state when robot telemetry has not updated for more than 5 seconds.

The UI does not need to be visually complex. A simple 2D canvas with a dot representing the robot is enough if the integration works correctly.

Communication Requirements

There are two separate communication boundaries in this assignment:

Boundary	Requirement
`Frontend <-> Cloud Backend`	`Browser-safe API such as REST, WebSocket, or similar`
`Cloud Backend <-> ROS Plugin`	`Candidate-designed robot-to-cloud communication`
`ROS Plugin <-> ROS Simulation`	`ROS topics using /pose, /battery_percentage, and /cmd`

You must clearly separate these boundaries in your code and README.

Do not hardcode localhost for communication between Docker containers. Use Docker service names and environment variables.

Docker Requirements

Provide Docker support for all services you build.

Required deliverables:

- docker-compose.yml for your services.
- Dockerfile for the ROS Plugin.
- Dockerfile for the Cloud Backend.
- Dockerfile for the Frontend.
- Clear instructions for running your stack together with the provided ROS simulation.

Your compose setup should make it clear which services are robot-side and which services are cloud-side, even if they run locally for the assignment.

All ports and service URLs must be configurable using environment variables.

README Requirements

Your README must include:

- Project architecture diagram or explanation.
- Technology stack used for each service.
- How to run the provided ROS simulation.
- How to run your Docker Compose stack.
- How to verify that the ROS Plugin is connected to ROS.
- How to verify that telemetry reaches the backend.
- How to verify that the frontend shows robot position and battery.
- How to verify that pressing W, A, S, or D moves the robot.
- Explanation of the communication protocol used between the ROS Plugin and Cloud Backend.
- Known issues or limitations.

Evaluation Criteria

Area	What We Look For
ROS integration	Correct subscription to /pose and /battery_percentage; correct publishing to /cmd
System boundary	ROS Plugin, backend, and frontend are separated correctly
Robot-to-cloud design	The communication method is justified and handles reasonable failure cases
Backend quality	Clear API, validation, health check, latest-state handling
Frontend functionality	Robot position, battery, keyboard control, stale state handling

Docker setup	Reproducible containers, environment variables, no container-to-container localhost assumptions
Code clarity	Readable structure, separation of concerns, useful logs
Documentation	README is accurate enough to run without asking questions

Deliverables

- Record a walkthrough video of your solution. The video should include your audio commentary and a screen recording of your running solution, explaining your approach and design decisions.
- A private GitHub repository containing your code.
- Invite `@billynugrahas` and `@alfianahar` as collaborators.
- A complete README as described above.
- All services are containerised and runnable with Docker.

Final Notes

We care more about a clean, working integration than a complicated UI.

If you choose a simple communication protocol, make sure it is implemented cleanly and explain its limitations. If you choose a more advanced protocol, make sure the added complexity is justified.

If you have questions, contact us at billy@movel.ai and alfian@movel.ai.

Only shortlisted candidates will be contacted.